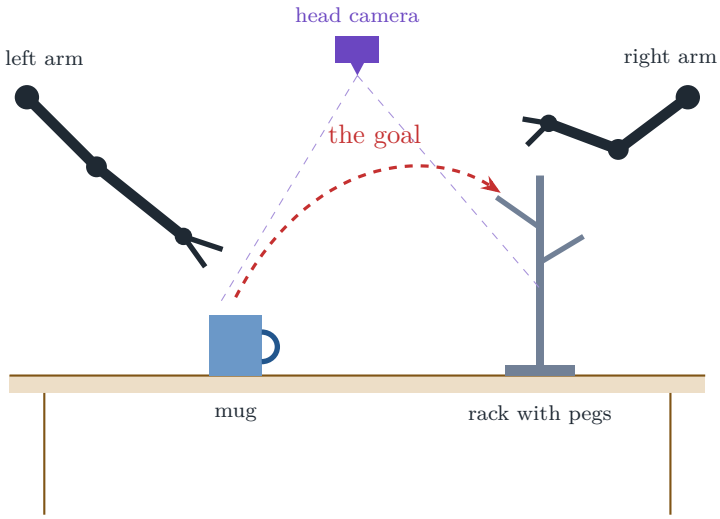
How Our Robot Learns to Hang a Mug

A friendly guide to the Agent-Aware VLA harness

Version 1.2 • October 2026



This guide explains the project from zero: what the robot does, who the "team" inside the software is, what information moves between the parts, how one episode runs, how the code is organised, how to change things, and how to set it up and run it, on your own computer or on rented cloud GPUs (Modal). Version 1.2 adds a beginner's map, a concrete turn-by-turn example, and explanations of what each setup command is doing. Coloured boxes help you find your way:

simple explanation | key idea | where to change it | careful

Covers branch `modal-simulator`. File and line references point to that code.

Contents

Start here: the five things to remember **3**

1 The story in one page **4**

2 Words you will see **5**

3 The three big pieces **6**
3.1 Piece 1: the simulator (the stage) . . . 6
3.2 Piece 2: the harness (the team) . . . 7
3.3 Piece 3: the cloud helpers . . . 8
3.4 Where Opus comes in . . . 8

4 The world the robot lives in **9**
4.1 The scene and the three cameras . . . 9
4.2 Fourteen numbers drive the robot . . . 9
4.3 Two kinds of time . . . 10

5 The task is split into three phases **10**

6 One turn of the loop **11**
6.1 A concrete example: one turn in plain English . . . 12
6.2 What happens after a skill finishes? . . . 13
6.3 Budgets: how much the robot may try . . . 13

7 The six move cards (skills) **14**
7.1 How a VLA call plays a chunk . . . 14
7.2 How a careful (analytic) move works . . . 15

8 House rules that keep things safe **15**
8.1 Three answers, not two . . . 15
8.2 The wall between public and private knowledge . . . 15
8.3 Opening a hand needs proof . . . 15
8.4 Other rules in one list . . . 16

9 The eyes: perception **16**
9.1 From a pixel to a 3D point . . . 17
9.2 Who looks at the pictures? . . . 17
9.3 Three points make a frame . . . 18
9.4 Is the mug lifted? Measuring against the table . . . 18
9.5 When a chosen pixel is bad . . . 19

10 Architecture: what runs where **19**
10.1 How the code layers depend on each other . . . 19

11 Where everything lives **20**
11.1 Which file do I open? . . . 21

12 Code tour: how it works today **22**
12.1 Starting an episode . . . 22
12.2 Choosing the coach . . . 22
12.3 What a move card looks like . . . 23
12.4 The heart of the turn loop . . . 23
12.5 Playing a policy chunk . . . 23

13 Cookbook: I want to. . . **24**

14 Configuration files **24**

15 How we work: the development ladder **26**

16 Setting up and running **27**
16.1 Common steps (both paths) . . . 27
16.2 Path A: run on your own GPU . . . 28
16.3 Path B: run on Modal . . . 29
16.4 Running cheaply . . . 30
16.5 Measured speed . . . 31
16.6 When something goes wrong . . . 31
16.7 What you get after a run . . . 32
16.8 Other ways to run . . . 32

17 Where things stand today **33**

18 Testing and contributing **33**

A Frequently asked questions **34**

Start here: the five things to remember

Like I'm five

You do not need to understand robotics before reading this guide. Read the project as a **conversation between programs**. One program takes a picture, another program describes what the picture seems to show, a coach chooses one move from a small menu, and ordinary Python code carries out that move safely. Then the programs look again. This repeats until the mug is hanging, something goes wrong, or a limit is reached.

1. **The simulator is the pretend physical world.** RoboTwin contains the table, mug, rack and robot. SAPIEN makes the objects obey gravity and draws the camera pictures.

2. **The policy is learned muscle memory.** The frozen Neotix VLA receives pictures and robot numbers and returns low-level actions. This project uses it; it does not retrain it.

3. **The planner is a coach, not a robot driver.** Opus or a scripted planner chooses one named skill such as `vla_act` or `move_to`. It cannot send arbitrary joint commands.

4. **The harness is the safety and translation layer.** It turns the chosen skill into simulator operations, checks limits and evidence, records what happened, and asks for a fresh observation.

5. **A run is one episode.** The episode starts with one seed and one reset. It ends with a summary such as `task_success`, `abort`, `step_budget`, or another recorded outcome. A new experiment normally means a new output directory.

Key idea

The most important boundary: the coach chooses *what kind of move* should happen; the executor decides *how to perform that move*. If you remember only one sentence, remember this one. It explains why the planner can be changed without rewriting the robot motion code.

How to read the arrows in this guide

An arrow labelled **observation** means "information going into a decision." An arrow labelled **action** or **move** means "a request to change the simulated robot." A **dashed arrow** usually crosses a machine boundary: the data is sent to Modal or an API. The same blue harness boxes can run on your laptop or inside the Modal episode container; only the computer providing the GPU changes.

1 The story in one page

Like I'm five

There is a robot with **two arms** standing at a table. On the table there is a **mug**. Next to it stands a little tree-shaped **rack** with pegs sticking out. The robot's job is to pick up the mug and **hang it on a peg by its handle**.
The robot already learned how to do this by *watching lots of examples*, a bit like a child copying a grown-up. That learned skill is called the **policy**. It is good at most of the job but often gets wobbly at the very end, when the handle has to slide onto the peg.
We are **not** allowed to teach the robot again. Instead, we add a **coach** who watches through the robot's cameras every few moments and says "keep going", "try this careful move instead", or "stop, this is not safe". The coach can only choose from **six move cards**. It never wiggles the robot's joints by itself.

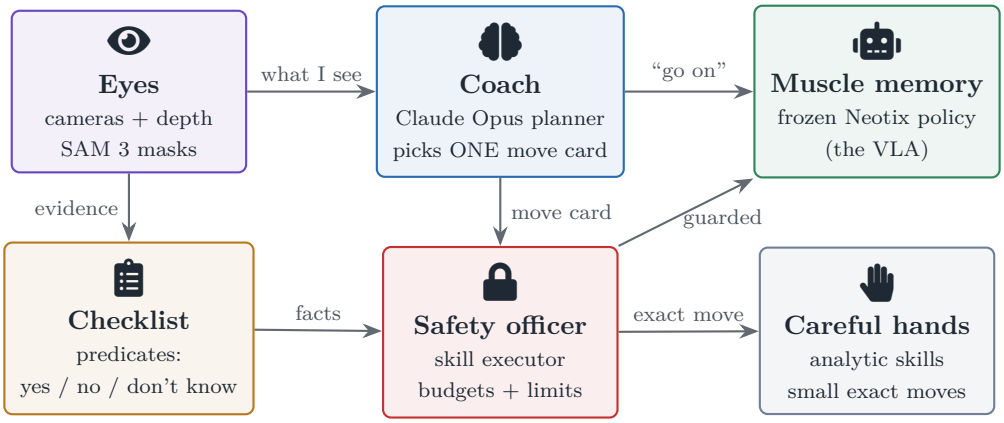


Figure 1: The "team" inside the software. Each box is a real part of the code.

Key idea

One move card per turn. The coach (Opus) reads the camera pictures and a few facts, then picks exactly one bounded skill. The software checks the card, the robot does it, and the coach gets fresh pictures. Nothing the coach says can move a joint directly.

Why is it built like this?

- **The policy is frozen.** It was trained elsewhere and is published as `Neotix-Robotics/robotwin-hanging-mug-dit`. We only *use* it.
- **Language models are bad at joint angles** but good at looking at pictures and judging "is this going well?". So Opus judges and chooses, and code does the exact maths.
- **Safety first.** Every move has hard limits (how far, how fast, how many tries). If anything is uncertain, the robot stops instead of guessing.

2 Words you will see

Word	Plain meaning
Joint	A robot "elbow" that can bend. Each arm has six.
Gripper	The robot's two-finger hand. `0` = closed, `1` = open.
TCP / end effector	The point between the fingers. Moves are described by where this point goes.
14-DoF	14 numbers that control both arms: 6 joints + 1 gripper per arm.
Policy / VLA	"Vision-Language-Action" model: pictures in, robot actions out. Ours is *frozen*.
Chunk	The policy answers with a short list of actions (e.g. 8) at once.
Simulator	A video-game-like world (RoboTwin) with real physics, so nothing breaks.
Episode / seed	One attempt at the task. The seed decides where the mug starts.
Observation	One snapshot: 3 camera pictures + the 14 numbers. Each has a sequence number.
RGB-D	Colour picture (RGB) plus Depth: how far away each pixel is.
Calibration	The camera's "lens settings" (intrinsic) and position (extrinsic). Needed to turn pixels into 3D points.
Mask / SAM 3	A cut-out showing which pixels are the mug, rack or table. SAM 3 makes them.
Planner	The coach. Either Claude Opus or a fixed script.
Skill	One of the six move cards (Section 7).
Predicate	A checklist question like "is the mug lifted?" Answer: `true`, `false` or `null` (unknown).
Anchor	A named 3D target such as `mug_pregrasp` or `rack_seat`, freshly computed each time.
Oracle	"Cheating" simulator knowledge (exact mug position). Only for debugging, never shown to the coach.
Harness	Our code around the frozen policy: the coach, the checklist, the move cards and the safety rules.
RoboTwin	The simulator task suite. Our task is `hanging_mug`.
SAPIEN	The physics engine and picture-drawing (rendering) engine under RoboTwin.
CuRobo	A GPU motion planner: turns "put the hand there" into a safe joint path.
Vulkan	The graphics driver language SAPIEN uses to draw pictures on an NVIDIA GPU.
Modal	A cloud service that rents GPUs by the second. Runs the policy, SAM 3 and, optionally, whole episodes.
Container	A fresh, clean rented computer that Modal starts for one job and throws away afterwards.
Volume	A cloud folder on Modal that keeps files between containers (assets, run results).
Secret	A password (API key or token) stored on Modal, never written in the code.

3 The three big pieces

Like I'm five

Think of a **puppet show**. The **stage** is the simulator: a pretend world where the table, the mug, the rack and the robot live, and where things fall and bump like in real life. The **team behind the stage** is the harness: our own code, which decides what the robot may do next and makes sure it is safe. Some team members are so big that they live far away on **rented computers in the cloud**: the policy (muscle memory), SAM 3 (the eyes that cut out objects) and Opus (the coach who talks).



Figure 2: The whole project is these three pieces talking to each other.

3.1 Piece 1: the simulator (the stage)

Like I'm five

The simulator is a **video game with real physics**. Nothing breaks, nothing costs money if it falls, and you can start again with a fresh table. The **seed** is like the dice roll that decides where the mug starts. Seed 3 always gives the same starting table.

It is made of layers, like a cake:

RoboTwin `hanging_mug` task	places mug, rack and robot; says "success" when the mug hangs
SAPIEN physics + renderer	gravity and contacts; draws camera pictures by *ray tracing* on the GPU (Vulkan)
CuRobo motion planner	"put the hand there" → a smooth joint path that avoids collisions
NVIDIA GPU with a Vulkan driver	your laptop, or a rented Modal GPU

Figure 3: The simulator stack. The pinned versions are in the README; our renderer tweak is `patches/robotwin-renderer.patch`.

Our code talks to the simulator through three small files:

- `harness/environment/robotwin.py` builds the task (`build_task`) and packs one observation: three pictures, optional depth, the 14 numbers and the success flag (`observation_of`).
- `development/simulator/local_simulator.py` (`LocalSimulator`) owns **one** RoboTwin task in this process: reset with a seed, then step.

- `harness/environment/local.py` (`LocalEnvironmentClient`) is what the skills use: `observe`, `step`, and for careful moves `plan` (CuRobo) and `tick` (one physics step).

Key idea

Pictures are the slow part. We measured each piece (Section 16.5): one physics step takes about 0.1 ms, a CuRobo plan about 0.3 s, but **drawing three camera pictures with depth takes about 0.8 s** on a laptop GPU. An episode draws hundreds of pictures, so a faster GPU (for example on Modal) helps the most.

Careful

RoboTwin changes the program's working folder when it starts, so run **one simulator per process**. Its renderer needs a working NVIDIA Vulkan driver; on laptops with two GPUs, tell Vulkan to use only NVIDIA (Section 16.6).

3.2 Piece 2: the harness (the team)

Like I'm five

The harness is the **rulebook plus the team** that stands between the coach and the robot. The coach may only hand in a move card. The team checks the card, does the exact maths, plays the move in small safe pieces, and writes everything down. The harness never teaches the robot anything new: the policy stays exactly as it was trained.

How it is set up today, in the order things happen:

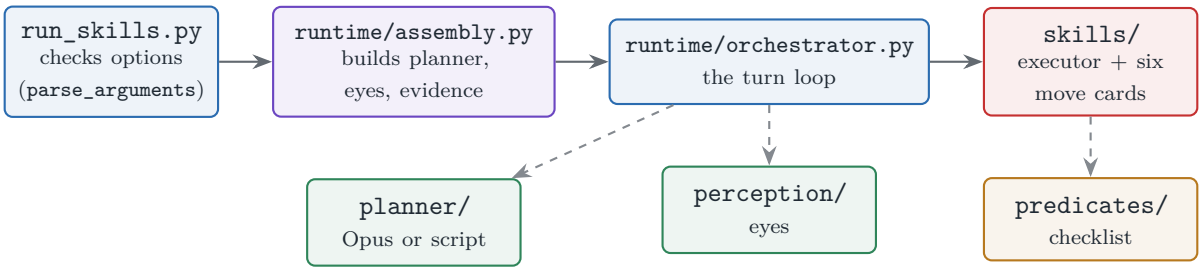


Figure 4: The harness in one picture. Section 6 explains the turn loop in detail.

Three switches decide *where the evidence comes from.* Pick exactly one of the first two (the third can be added to the second):

Evidence mode	What it means	Use it for
`--development-oracle`	The simulator secretly tells the checklist the true mug position. Opus never sees it.	debugging skills; cheapest runs
`--perception-factory ...modal_sam3:create_detector`	Real "eyes": SAM 3 masks + depth + Opus keypoints. What a real robot could know.	honest full-task runs
`--planner-pixel-grounding`	Opus also points at pixels for targets inside its move card.	extra fallback for anchors

3.3 Piece 3: the cloud helpers

Helper	Code	Runs on	What it does
Neotix policy	`modal_apps/policy.py`	Modal T4	pictures + 14 numbers in, a chunk of actions out. Frozen.
SAM 3	`modal_apps/sam3.py`	Modal T4	cuts out mug, rack and table in one RGB picture.
Claude Opus	`harness/planner/anthropic.py`	Anthropic API	the coach; see the next subsection.
Episode runner *(new)*	`modal_apps/simulator.py`	Modal L4	runs a *whole* episode (simulator + harness) in the cloud. Optional.

Idle helpers switch themselves off (policy after 10 minutes, SAM 3 after 2), so they only cost money while used.

3.4 Where Opus comes in

Like I'm five

Opus is the coach who **looks and talks** but never touches the robot. It has up to five jobs. The first is always on when you choose `--planner anthropic`; the others are switched on by the config or a flag. Every job sends Opus only public pictures and facts, and every answer is checked by code before it is used.

Opus job	When	Code	Switched on by
1. Pick the next move card	every turn	`AnthropicPlanner.select` (`planner/anthropic.py`)	`--planner anthropic`
2. Find the rack peg (three keypoints)	once, after reset	`assembly.initialize_rack_frame`, `perception/opus_frame.py`	`perception.opus_rack_grounding`
3. Find the mug handle (three keypoints)	when a hang target is needed	`OpusObjectFrameGrounder` (`opus_frame.py`)	needs job 2
4. Point at the mug's bottom for lift height	when lift is checked	`OpusMugSupportGrounder` (`perception/table_plane.py`)	`perception.opus_mug_lift_grounding`
5. Choose target pixels inside its card	when a move needs a target	`PlannerPixelAnchorPreparer` (`perception/preparation.py`)	`--planner-pixel-grounding`

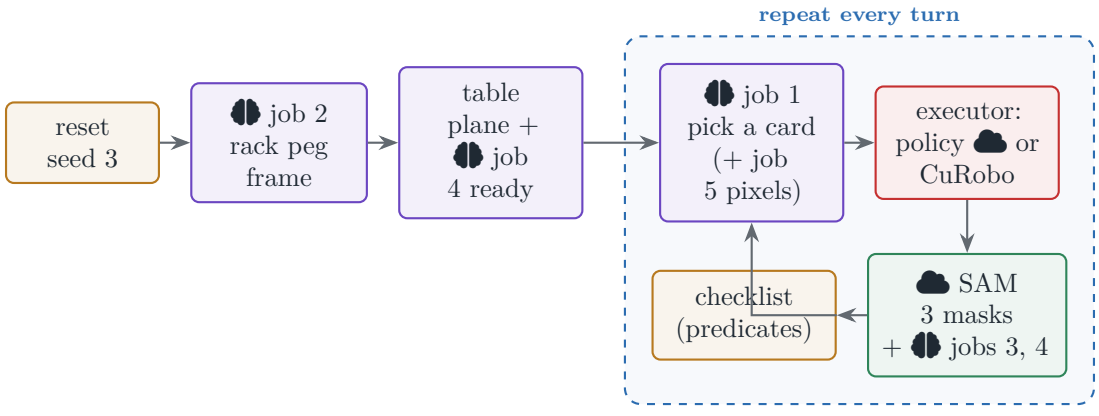


Figure 5: One full-task episode in time. Purple boxes are moments when Opus is asked something.

Careful

Opus calls cost money and take seconds (about 12 s per planner turn in our runs). The config limits them (`anthropic.max_calls`) and names the model explicitly (`claude-opus-5`); there is no fallback model. Simulator-only facts are filtered out before anything reaches Opus.

4 The world the robot lives in

4.1 The scene and the three cameras

The simulator is **RoboTwin** running the `hanging_mug` task. The robot has three cameras. Every observation contains all three pictures.

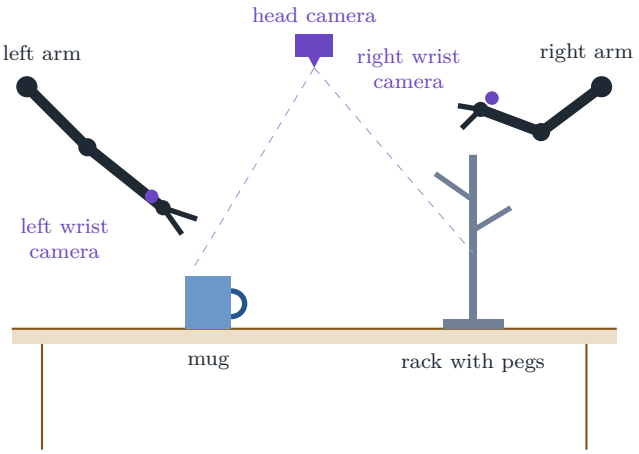


Figure 6: The scene. Camera names in the code: `head_camera`, `left_camera`, `right_camera`.

4.2 Fourteen numbers drive the robot

The policy reads the pictures plus 14 numbers, and answers with 14 new numbers per step.

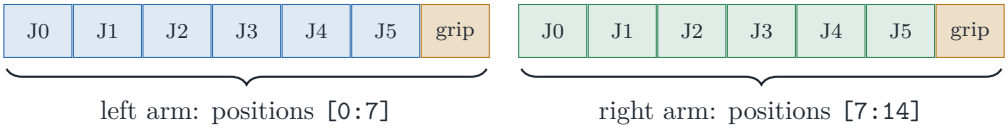


Figure 7: The 14-number action layout, defined once in `harness/environment/layout.py`.

4.3 Two kinds of time

- **Policy actions**: one action = one call to RoboTwin's `take_action`, which itself runs many physics steps. The policy sends actions in chunks.
- **Physics ticks**: the careful analytic skills move the arm one tick at a time. Physics runs at **250 ticks per second**, so 125 ticks is half a second.

5 The task is split into three phases

Like I'm five

Hanging a mug is hard to do in one go, so the robot does it in three chapters. The left hand picks up the mug. Then it puts it back down nearby so the right hand can take it. Then the right hand picks it up and hangs it on the rack. You can only go forward to the next chapter, never back.

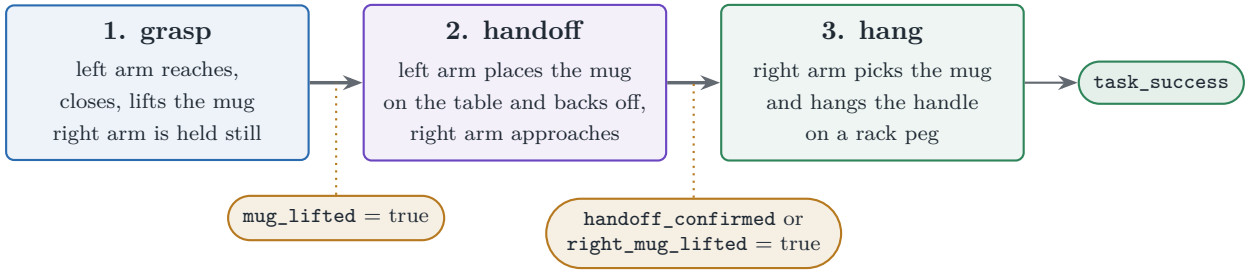


Figure 8: Phases and the public milestones that unlock the next phase.

Key idea

A **true public milestone** sets the minimum phase. When the cameras cannot tell (the answer is `null`), Opus may move the phase forward by its own visual judgement, but only **one step per turn**. Handoff starts once the mug is lifted (or already staged or transferred). Hang needs proof that the *right* arm actually has the mug (`handoff_confirmed` or `right_mug_lifted`); a mug merely placed on the table is not enough. See </> `harness/planner/anthropic.py:372` (`_public_phase_floor`) and `_resolve_phase`.

6 One turn of the loop

Every episode is a loop of **turns**. One turn = look, decide, check, do, record.

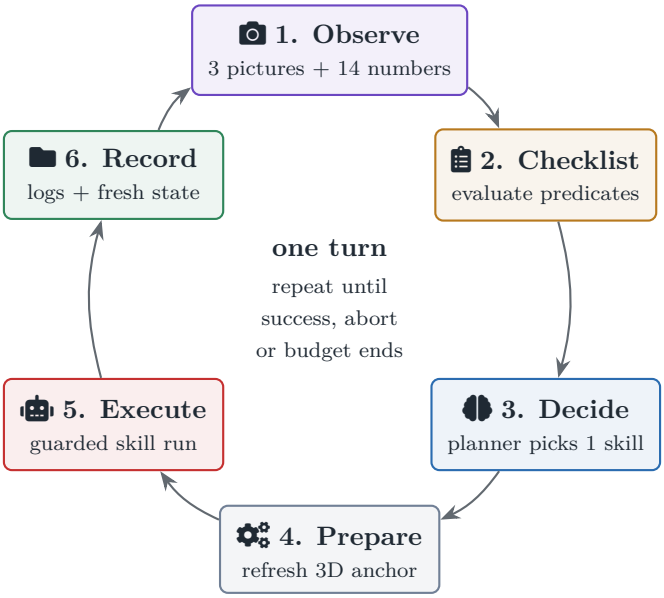


Figure 9: The turn loop, implemented in `SkillOrchestrator.run` (</> `harness/runtime/orchestrator.py:224`).

6.1 A concrete example: one turn in plain English

Like I'm five

Imagine that the last turn already closed the left gripper around the mug. The next turn does not blindly continue from memory. It starts by looking again. This is important because the mug may have moved, the fingers may have slipped, or the camera may be uncertain.

1. **Observe:** RoboTwin returns three RGB pictures and the current 14-number robot state. This is the new "what is happening now?" snapshot.
2. **Checklist:** the evidence code answers questions such as `mug_grasped` and `mug_lifted`. Each answer is `true`, `false`, or `null`; `null` means "we do not have enough evidence," not "false."
3. **Decide:** the planner sees the allowed pictures and facts and returns one card, for example `move_to(target="mug_lift")` or `vla_act`.
4. **Prepare:** if the card names a 3D target, perception converts a camera pixel and its depth into a fresh world-space anchor. The code does not trust an old remembered position.
5. **Execute:** the executor validates the card, applies its limits, and either asks the frozen policy for a short action chunk or lets CuRobo plan a careful analytic move.
6. **Record:** the result, checks, timing and any failure flag are written to the run files. The next turn starts with a new observation, not with an automatic replay.

Key idea

The planner does not receive a live video stream and it does not continuously steer the robot. It receives a snapshot, makes one bounded decision, waits for the executor to finish or stop, and then receives another snapshot. This is why the guide calls the system a **turn loop**.

6.2 What happens after a skill finishes?

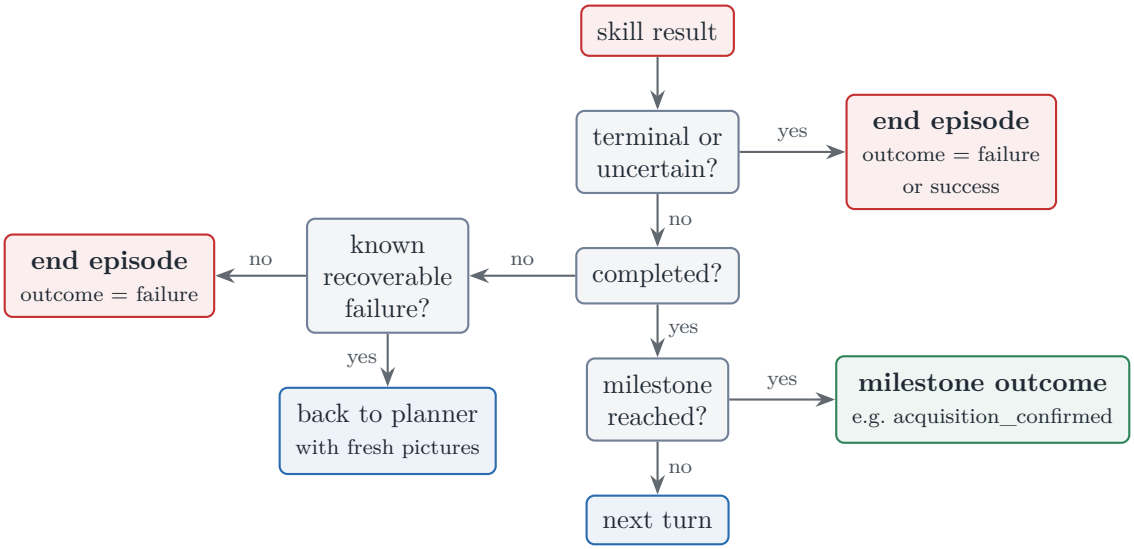


Figure 10: Turn outcome decisions. "Recoverable" failures are a short, explicit list in `recovery_flags` (</> `harness/runtime/orchestrator.py:144`).

The recoverable failures are: a move that was rejected *before* moving, a move that moved but missed its target, a contact between the two arms, and a temporary perception gap. In every case the planner sees fresh pictures; **motion is never replayed automatically**.

6.3 Budgets: how much the robot may try

Budget (`EpisodeBudget`)	Default	Meaning
`max_skill_calls`	100	move cards per episode
`max_policy_actions`	1200	policy actions executed in total
`max_vla_chunks`	150	times the policy may be asked
`max_analytic_ticks`	12000	physics ticks for careful moves (48 s)
`max_turns` (`RunSettings`)	20	planner turns

</> `harness/skills/schemas.py:123`. Configs usually override these.

7 The six move cards (skills)

Skill	What it does	Arms	Limits
`vla_act`	Ask the frozen policy for short chunks and play them, with checks after every action.	both	≤32 chunks, ≤24 actions/chunk
`move_to`	Move one hand to a fresh named target (or current pose) plus a small offset.	one	offset ≤ 20 cm
`rotate_wrist`	Turn one wrist around the vertical axis.	one	≤ 30°
`set_gripper`	Slowly open/close one hand. Opening needs permission.	one	50–500 ticks ramp
`release`	Fully open one hand, only when support is proven.	one	guarded
`abort`	Stop the episode. No motion.	none	–

`harness/skills/schemas.py:13`. Blue skills are *analytic*: exact maths, no learning.

7.1 How a VLA call plays a chunk

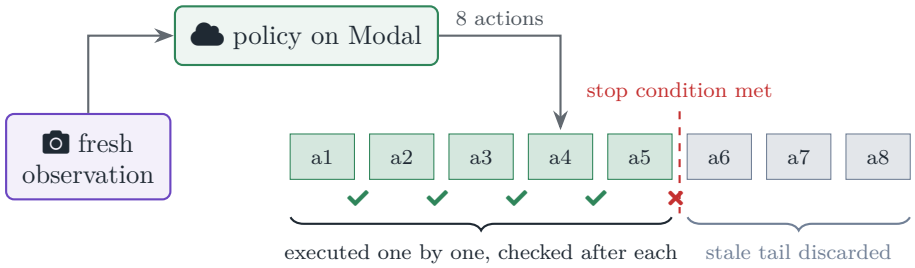


Figure 11: One chunk. After every action the code checks arm-to-arm contact, the task checker and cheap stop predicates. Camera-based checks (SAM 3, Opus) run once, when the call hands control back to the planner. Unused actions are thrown away, never executed later (`harness/skills/vla_act.py:165`).

Key idea

Feedback boundary. In Opus mode the policy first gets an uninterrupted window at the start of each phase (full-task config: 64 actions for grasp, 128 for handoff, 64 for hang, set by `anthropic.*_initial_feedback_after_actions`). After that, each `vla_act` call ends after `planner_feedback_chunks` chunks (normally 1 chunk of 16 actions). The coach then sees new pictures and decides again. When RoboTwin's checker first reports success, the policy may finish a fixed 16-action follow-through so the gripper can let go cleanly (`NATIVE_SUCCESS_FOLLOW_THROUGH_ACTIONS`).

7.2 How a careful (analytic) move works

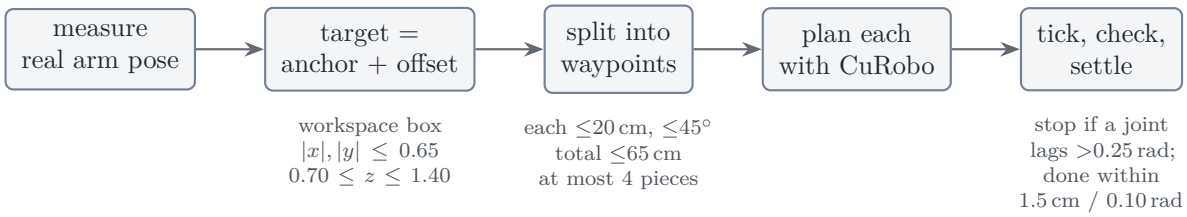


Figure 12: Analytic skills always start from the *measured* pose, never a remembered one (</> `harness/skills/analytic.py`).

8 House rules that keep things safe

8.1 Three answers, not two

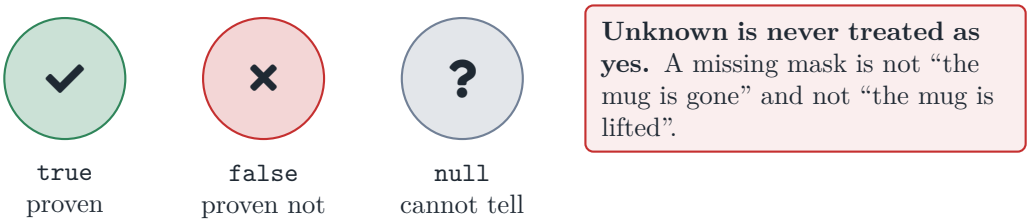


Figure 13: Predicate answers (`PredicateResult`, </> `harness/predicates/base.py`).

8.2 The wall between public and private knowledge

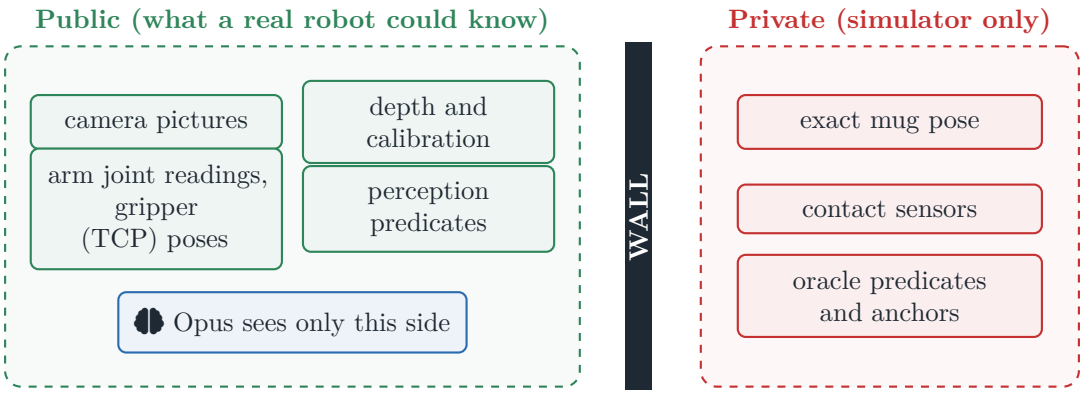


Figure 14: The oracle may gate execution during debugging (`--development-oracle`), but it is filtered out before anything is sent to Opus (</> `harness/planner/prompt_builder.py`).

8.3 Opening a hand needs proof

The robot may only open a hand that is holding the mug when the public checklist says that `release_ready` is `true` (the mug is resting on the table), or, at the rack, that `rack_seat_aligned` is `true`. The planner enforces this before anything moves:

Listing 1: harness/planner/anthropic.py:236

```
release_authorized = self._public_predicate_true(context, "release_ready")
rack_seat_authorized = (vla_phase == "hang" and
    self._public_predicate_true(context, "rack_seat_aligned"))
release_allowed = (release_authorized or
    (command.action.value == "release"
     and command.parameters.authorization == "rack_seat_alignment"
     and rack_seat_authorized))
if (not release_allowed and
        (command.action.value == "release"
         or (command.action.value == "set_gripper" and command.parameters.value > 0.0))):
    raise InvalidPlannerResponse("release_not_publicly_authorized")
```

8.4 Other rules in one list

- **Never retry an uncertain command.** If a response is lost after sending an action, the episode stops.
- **Anchors expire.** A target is valid only for the observation it was computed from.
- **Arm contact recovery is narrow**: right hand moves 3–10 cm away, at most 2 retreats and 3 incidents.
- **Grounding retries are limited**: a rejected pixel can be re-chosen at most twice, the second time from a different camera.
- **Losing sight is not a reason to stop**: if the cameras lose the mug and table (or mug and rack while hanging) during a policy phase, the runtime itself continues the policy for one bounded chunk and looks again, instead of letting Opus abort (`_perception_gap_vla_continuation` in `planner/anthropic.py`).
- **Logs never contain secrets** or raw provider text; only short error codes.

9 The eyes: perception

Like I'm five

A picture is flat. To reach for the mug, the robot needs to know *where in 3D* it is. Each camera also tells how far away every pixel is (depth). If you know which pixel is the mug and how far away that pixel is, and where the camera is, you can work out the mug's 3D position.

9.1 From a pixel to a 3D point

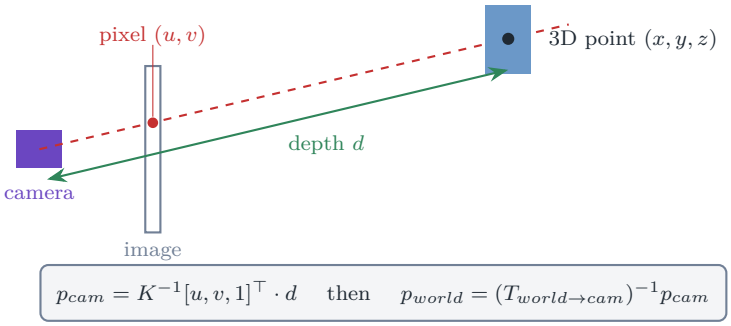


Figure 15: Unprojection. K is the intrinsic matrix, T the extrinsic pose.

Listing 2: </> harness/perception/rgbd.py:97 `RGBDCamera.unproject` (abridged)

```
depth = self.sample_depth(u, v, radius=radius)     # local median, ignores zeros
pixel = np.array([float(u), float(v), 1.0])
camera_xyz = np.linalg.solve(self.intrinsic, pixel) * depth
camera_h = np.append(camera_xyz, 1.0)
world = np.linalg.solve(self.world_to_camera, camera_h)
return tuple(float(value) for value in world[:3] / world[3])
```

9.2 Who looks at the pictures?

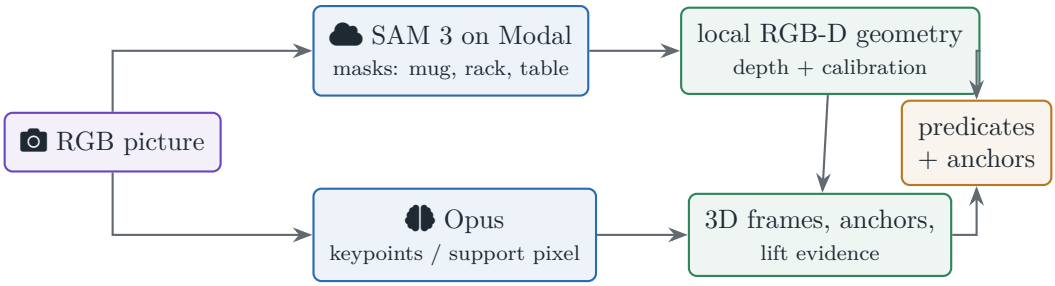


Figure 16: Only RGB images leave the machine. Depth maths always stays local.

- **SAM 3** (`harness/perception/modal_sam3.py`) draws masks. Mask confidence floors: mug 0.40, rack 0.60 (0.18 with the exploration prompt "mug rack"), table 0.60.
- **Opus keypoints** (`opus_frame.py`) give three ordered points on the rack peg or the mug handle; three points define a little 3D coordinate frame. The points are turned into 3D with depth alone (each must sit on a smooth depth surface); no SAM mask is needed (`ground_public_object_frame_rgbd`).
- **Opus support pixel** (`table_plane.py`) marks the bottom of the mug, used to measure lift height.

9.3 Three points make a frame

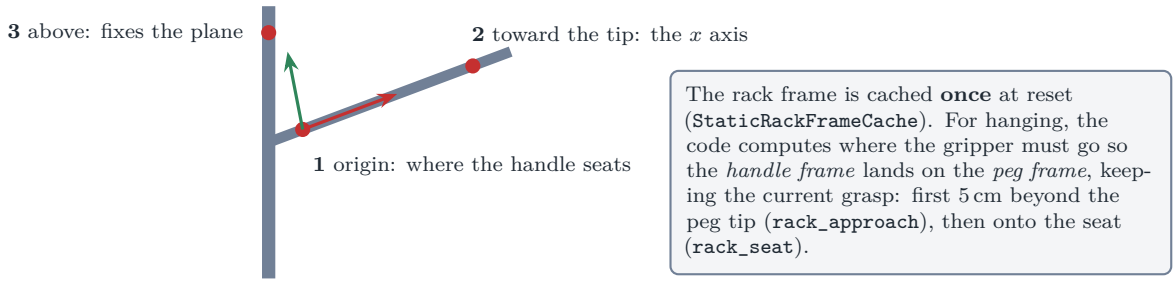


Figure 17: Keypoints become frames (</> harness/perception/pose_grounding.py, hang_targets.py).

9.4 Is the mug lifted? Measuring against the table

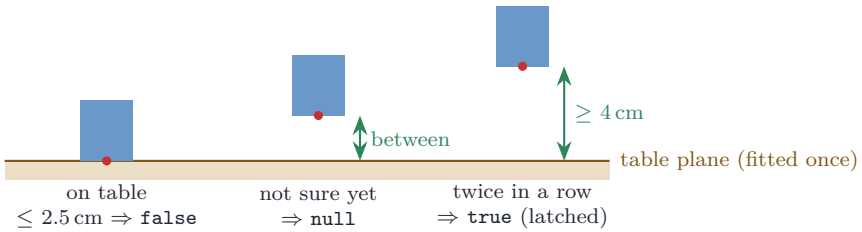


Figure 18: Table-relative lift evidence (MugLiftEvidenceTracker, </> harness/perception/table_plane.py:196).

Where to change it

Lift thresholds are constructor arguments: `lifted_distance_m=0.04`, `table_distance_m=0.025`, `consecutive_observations=2` in `harness/perception/table_plane.py`. The first lift check waits until the policy has run `perception.mug_lift_first_check_after_actions` actions (64 in the full-task config); before that the answer is `null`. Mask confidence floors are `MUG_MASK_MIN_CONFIDENCE` and `EXPLORE_RACK_MASK_MIN_CONFIDENCE` in `harness/runtime/assembly.py`. Prompts for Opus perception live in `harness/prompts/`.

9.5 When a chosen pixel is bad

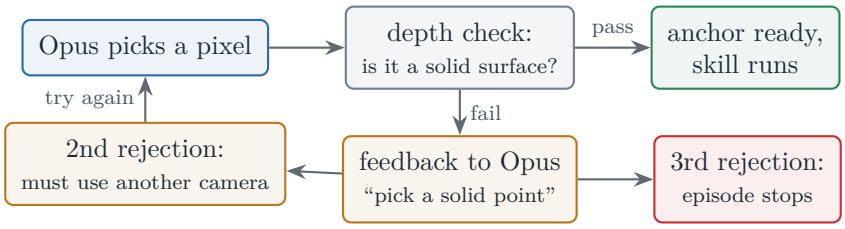


Figure 19: Grounding reselection (`MAX_GROUNDING_RESELECTIONS = 2`, `harness/runtime/orchestrator.py:19`).

10 Architecture: what runs where

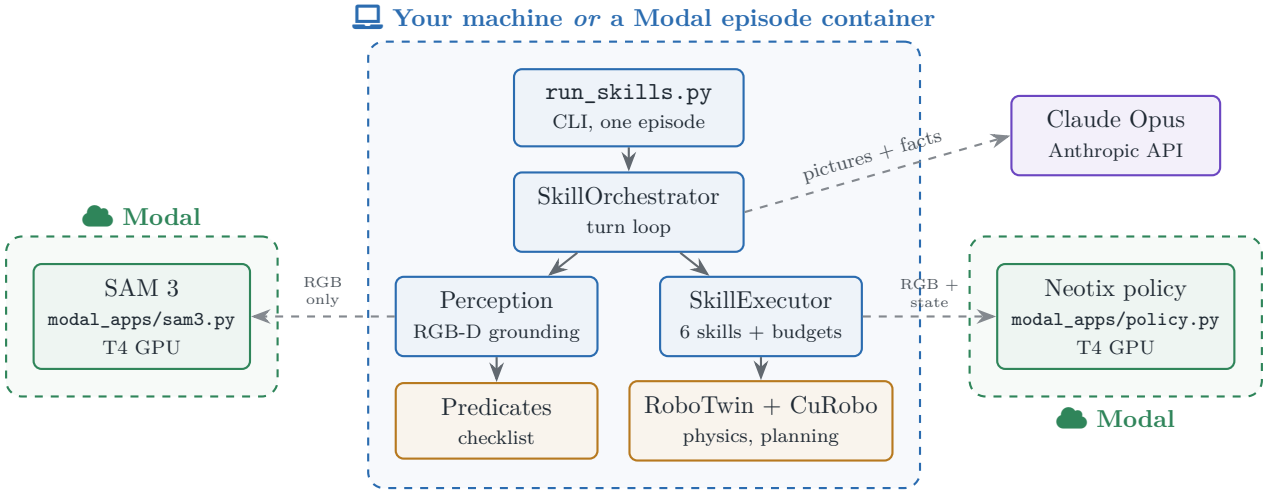


Figure 20: System architecture. Dashed arrows cross the network. The blue box runs either on your own GPU or, unchanged, inside a Modal container (Section 16.3).

10.1 How the code layers depend on each other

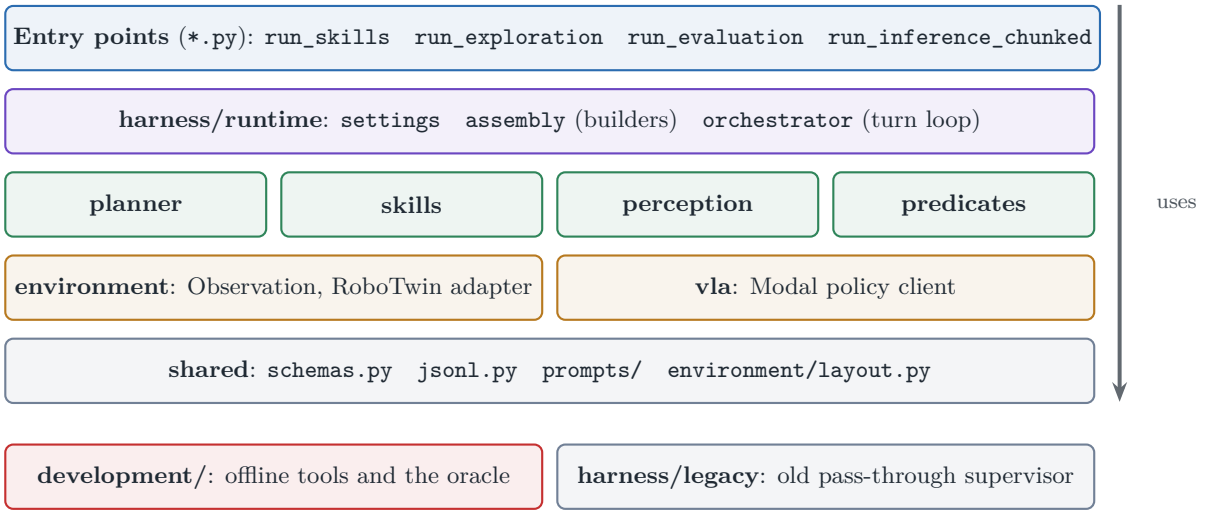


Figure 21: Higher layers use lower layers, never the other way round. `development/` is used only by the CLIs and tests.

11 Where everything lives

```
Agent-Aware-Vision-Language-Action-Model/
├─ run_skills.py              one supervised episode
├─ run_exploration.py         many episodes, one process each
├─ run_evaluation.py          frozen, fair evaluation
├─ run_inference_chunked.py   baseline: policy alone
├─ harness/
│   ├─ runtime/               settings, builders, turn loop
│   ├─ planner/               Opus + scripted coaches, prompt, parser
│   ├─ skills/                the six move cards
│   ├─ perception/            eyes: masks, depth, frames
│   ├─ predicates/            the checklist
│   ├─ environment/           observation, RoboTwin adapter
│   ├─ vla/                   talk to the policy on Modal
│   ├─ memory/  exploration/  evaluation/
│   ├─ prompts/               text sent to Opus
│   └─ legacy/                old pass-through supervisor
├─ modal_apps/
│   ├─ policy.py  sam3.py     cloud helpers
│   ├─ simulator.py           cloud image, spike, remote episodes
│   └─ episode_request.py     turns CLI options into a remote run
├─ development/
│   ├─ simulator/  replays/  capture/
│   └─ checkpoints/  recovery/  demos/
├─ configs/                   YAML presets
└─ requirements/  tests/  patches/
```

Figure 22: Repository layout. `RoboTwin/`, `data/` and `docs/` are local and not in git.

11.1 Which file do I open?

I want to understand...	Open
how an episode is started and cleaned up	`run_skills.py`
how the parts are built and connected	`harness/runtime/assembly.py`
the turn-by-turn loop	`harness/runtime/orchestrator.py`
how Opus is asked and checked	`harness/planner/anthropic.py`
what Opus is told	`harness/prompts/common.md`, `full_task.md`
what a move card may contain	`harness/skills/schemas.py`
how the policy chunk is played	`harness/skills/vla_act.py`
how careful moves are executed	`harness/skills/analytic.py`
how pictures become 3D anchors	`harness/perception/grounding.py`, `preparation.py`
the checklist logic	`harness/predicates/`
how RoboTwin is started	`harness/environment/robotwin.py`
the cloud model code	`modal_apps/policy.py`, `sam3.py`
how an episode runs on Modal	`modal_apps/simulator.py`, `episode_request.py`
how the simulator is wrapped	`development/simulator/local_simulator.py`, `harness/environment/local.py`

12 Code tour: how it works today

12.1 Starting an episode

`main()` checks the command-line options with `parse_arguments()` *before* anything expensive starts (the Modal launcher reuses the same check on your machine), then `run_episode()` owns the simulator for exactly one reset:

Listing 3: `run_skills.py:200` `run_episode` (abridged)

```
planner = assembly.build_planner(args.planner, settings, planner_settings=planner_settings,
                                 pixel_grounding=args.planner_pixel_grounding)
simulator = LocalSimulator(args.robotwin_root.resolve(), rgbd_enabled=...)
environment = open_environment(simulator, video=video, frozen=frozen)
initial_observation = environment.reset(args.seed)  # Exactly one reset, owned by this runner.
require_camera_frames(initial_observation, required_cameras)  # RGB-D runs: needed cameras present

if settings.perception.opus_rack_grounding:          # optional: ground the rack once
    rack_frame_cache, rack_grounding = assembly.initialize_rack_frame(...)
if settings.perception.opus_mug_lift_grounding:      # optional: table-plane lift evidence
    assembly.initialize_mug_lift_evidence(...)
predicates, prepare = build_evidence(...)             # public perception, pixels or oracle
executor = SkillExecutor(environment, ModalVLAClient(), budget=settings.budget, ...)
controller = SkillOrchestrator(executor, planner, prepare=prepare, ...)
summary = controller.run()
```

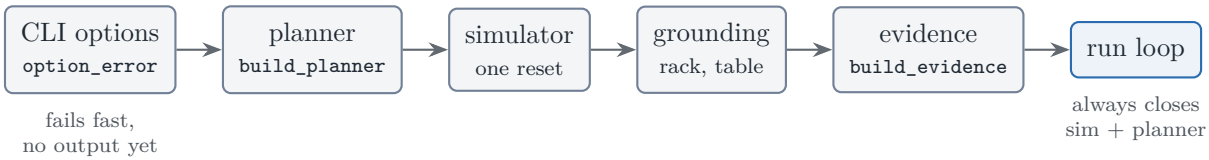


Figure 23: Episode start-up order.

12.2 Choosing the coach

Listing 4: `harness/runtime/assembly.py:92`

```
92  def build_planner(kind, run_settings, *, planner_settings=None, pixel_grounding=False):
93      """Return the planner for ``kind`` ("anthropic" or "scripted")."""
94      if kind == "anthropic":
95          return AnthropicPlanner(planner_settings, require_pixel_grounding=pixel_grounding)
96      if kind != "scripted":
97          raise ValueError("unsupported_planner")
98      if run_settings.handoff is not None:
99          return HandoffPlanner(run_settings.recipe, run_settings.handoff)
100     return GraspRetryPlanner(run_settings.recipe)
```

Where to change it

New planner? Write a class with `select(context) -> PlannerSelection` (protocol in `harness/planner/base.py`) and add one branch here. The loop never needs to know which planner it talks to.

12.3 What a move card looks like

Listing 5: harness/skills/schemas.py:84 (abridged)

```python
class SkillCommand(Model):
    action: SkillType                                  # one of the six skills
    arm: Literal["left", "right", "both", "auto"] = "auto"
    parameters: VLAParameters | MoveParameters | ...   # typed per skill
    stop: StrictStr | None = None                      # a predicate name, for vla_act
    reason: Annotated[StrictStr, Field(max_length=1000)] | None = None
```

Every card is validated twice: once when Opus answers (`response_parser.py`) and again inside the executor. Unknown fields are rejected (`extra="forbid"`).

12.4 The heart of the turn loop

Listing 6: harness/runtime/orchestrator.py:268 (abridged)

```python
selection = self.planner.select(PlannerContext(turn, observation.copy(), predicates, ...))
command = SkillCommand.model_validate(selection.command.model_dump())
self.log({**base, "event_type": "planner_selection", ...})   # intent logged before motion

prepared, rejection_code = self._prepare_anchor(command, observation, camera_to_avoid)
if rejection_code is not None:          # bad grounding: give feedback, try again
    grounding_feedback, grounding_camera_to_avoid = reselection_feedback(command, n)
    continue

previous = executor.execute(command)    # the only place robot motion starts
failure = turn_failure(command, previous, post)
flags = recovery_flags(previous)        # known, recoverable failure types
```

12.5 Playing a policy chunk

Listing 7: harness/skills/vla_act.py:68 (abridged)

```python
while context.chunks < (success_chunk_limit or configured_chunk_limit):
    observation = env.observe()
    context.chunks += 1                           # failed requests also count
    actions = np.asarray(context.vla.infer(observation, parameters.chunk_size))
    # ... validate the whole chunk (shape 14, finite) before ANY motion ...
    for action in actions:
        stopped = predicate_stop()                # e.g. mug_lifted reached?
        if stopped:
            return stopped                        # remaining actions are discarded
        context.observation = env.step(action)
        if overlap_check():                       # the two arms touched
            return "arm_overlap_detected"
```

13 Cookbook: I want to...

I want to...	Do this
change what Opus is told	Edit `harness/prompts/common.md` or `full_task.md`. The prompt hash in `manifest.json` changes, so old and new runs stay distinguishable.
change budgets or chunk sizes	Edit the YAML in `configs/` (`budget:`, `anthropic.vla:`). No code change.
get feedback after more chunks	`anthropic.planner_feedback_chunks` (1–4) in the config; the first window of each phase is `anthropic.<phase>_initial_feedback_after_actions`.
use another segmentation model	Write a factory returning an object with `detect(images, *, observation_sequence)` and pass `--perception-factory pkg.module:factory`.
add a new planner	Implement `select`, register it in `assembly.build_planner`, add a CLI choice in `run_skills.build_parser`.
use another policy backend	Implement `reset_history()` and `infer(observation, n)` (`harness/vla/base.py`).
add a checklist question	Add the name to `PREDICATE_NAMES` (`predicates/registry.py`) and teach a provider to answer it; return `null` when unsure.
tune lift or mask thresholds	See the green box in Section 8.
add a new skill	Rare and risky: add a `SkillType`, parameter model, implementation, registry entry, tool description, and tests. Talk to the team first.
ask Opus a new perception question	Build it with `harness/perception/vision_request.py` and add a prompt file.

Careful

Do not send simulator-only facts to Opus, do not remove the release check, and do not let a failed command be retried automatically. These rules are what make results trustworthy.

14 Configuration files

A config is a YAML file read into `RunSettings` (</> `harness/runtime/settings.py`). This is the richest preset, `configs/skill_runtime_opus_full_task_rgbd_table_lift.yaml`:

Listing 8: An annotated full-task configuration

```yaml

record_video: true              # writes episode.mp4 into the run folder
max_turns: 100                  # planner turns
perception:
  rgbd_enabled: true            # capture depth + calibration
  opus_rack_grounding: true     # ground the rack once at reset
  opus_mug_lift_grounding: true # table-plane lift evidence
  mug_lift_first_check_after_actions: 64  # no lift check before this
  rack_grounding_camera: right_camera
  rack_prompt_profile: explore_mug_rack
budget:
  max_skill_calls: 100
  max_analytic_ticks: 2500
  max_policy_actions: 800
  max_vla_chunks: 100
anthropic:
  model: claude-opus-5          # explicit; there is no fallback model
  task_scope: full_task         # grasp -> handoff -> hang
  planner_feedback_chunks: 1    # after the first window, Opus looks after every chunk
  grasp_initial_feedback_after_actions: 64     # uninterrupted policy actions
  handoff_initial_feedback_after_actions: 128  #   at the start of each phase
  hang_initial_feedback_after_actions: 64
```

```
  vla: {max_chunks: 32, chunk_size: 16, max_policy_actions: 768}
```

Config family	Use
`skill_runtime.yaml`	small scripted acquisition (no Opus); good first run
`skill_runtime_rgbd.yaml`	same, with depth for a detector
`skill_runtime_left_transfer.yaml`, `skill_runtime_full_handoff.yaml`	scripted handoff experiments
`skill_runtime_opus_*.yaml`	Opus planner, from acquisition to full task
`explore_*.yaml`	campaigns for `run_exploration.py`
`evaluation.yaml`	frozen evaluation seeds for `run_evaluation.py`
`harness.yaml`, `baseline.yaml`	legacy pass-through hook for the baseline

15 How we work: the development ladder

Like I'm five

Learning to swim, you start in the bathtub, then the shallow end, and only then the deep end. We test changes the same way. Each rung up the ladder costs more time and money, so only climb when the rung below passes.

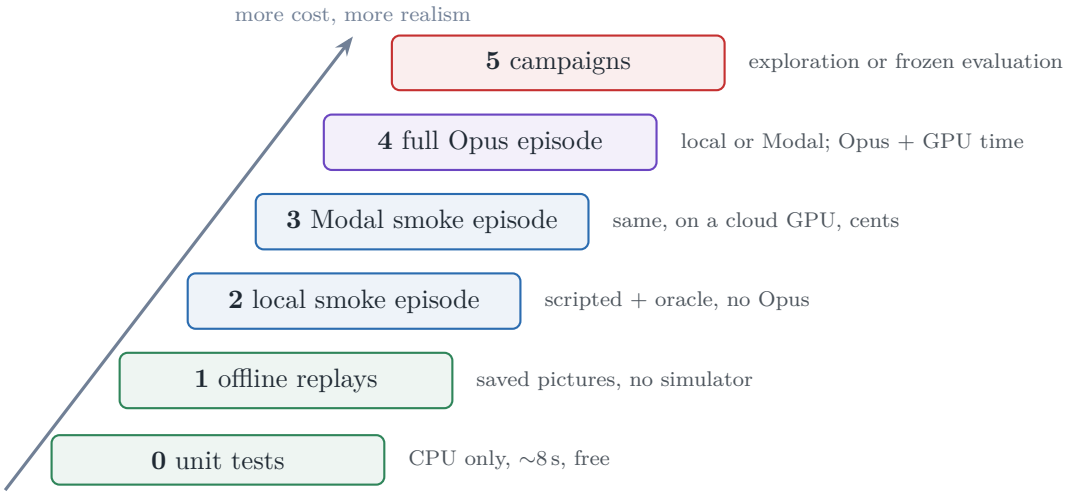


Figure 24: Climb one rung at a time. Most bugs are cheapest to find on rungs 0–2.

Rung	Command (from the repository root)
0 tests	`python -m unittest discover -s tests -q`
1 replays	`python -m development.replays.perception --help` (and the other tools in `development/replays/`)
2 local smoke	`python run_skills.py --config configs/skill_runtime.yaml --development-oracle --output data/runs/smoke_01`
3 Modal smoke	`modal run modal_apps/simulator.py::episode --config configs/skill_runtime.yaml --development-oracle --output data/runs/modal_smoke_01`
4 full episode	Section 16.2 (local) or Section 16.3 (Modal)
5 campaigns	`run_exploration.py`, `run_evaluation.py` (local only for now)

Key idea

The rung-2 smoke run is *expected* to stop with `"outcome": "step_budget"`: the scripted recipe gives its first move 1000 physics ticks, and the planned path needs a little more. It still proves the simulator, CuRobo and logging work. Same result on Modal = the remote runner behaves like the local one.

16 Setting up and running

There are two ways to run an episode. Both use the same command-line options and produce the same run folder; only *where the simulator runs* changes.

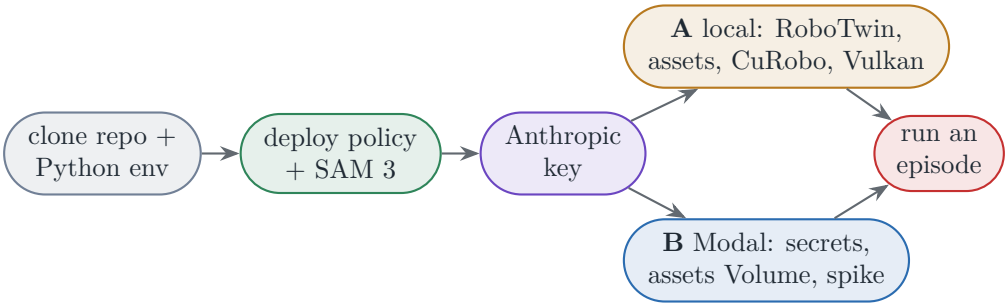


Figure 25: Common steps, then path A (your own NVIDIA GPU) or path B (rented Modal GPU). You can set up both.

	A: local	B: Modal
You need	Linux, NVIDIA GPU with Vulkan, ~10 GB disk	any machine with Python and a Modal account
Speed	pictures ~0.8 s each on a 4 GB laptop GPU	pictures ~0.11 s each on an L4
Costs	electricity; Opus + policy calls	GPU time (~$0.6–0.8/h) + Opus + policy calls
Good for	debugging with the simulator in reach	long or many runs, small laptops

16.1 Common steps (both paths)

Like I'm five

The commands below prepare three separate helpers. The Python environment holds the project's code and libraries. The policy and SAM 3 deployments put the learned model and object detector on Modal. The Anthropic variable gives the coach permission to answer. None of these commands runs an episode yet; they only prepare the pieces needed by a later episode command.

```
git clone <this repository> && cd Agent-Aware-Vision-Language-Action-Model
conda create -n robotwin python=3.10 -y && conda activate robotwin
python -m pip install -r requirements/sim.txt -r requirements/planner.txt 'modal>=1,<2'

modal setup                                  # log in to Modal once
modal deploy modal_apps/policy.py            # frozen Neotix policy (T4)

# SAM 3:  first request access at huggingface.co/facebook/sam3 and wait for approval
modal secret create huggingface HF_TOKEN=hf_...    # add --force to replace it
modal deploy modal_apps/sam3.py
modal run modal_apps/sam3.py                 # smoke test:  prints "SAM 3 ...  ready"

# Anthropic key, kept inside the conda env (never in git)
conda env config vars set ANTHROPIC_API_KEY=sk-ant-...
conda deactivate && conda activate robotwin
```

- **First three lines:** download the repository, create an isolated Python 3.10 environment, and install the simulator/planner/Modal client libraries.
- **`modal setup`:** authenticate the Modal command-line tool. This is not the Anthropic login and does not upload the repository by itself.

- **`modal deploy ...policy.py`:** start or update the persistent remote policy helper. The episode runner later calls this helper with an observation.
- **SAM 3 lines:** store the Hugging Face token as a Modal secret, deploy the detector, and run its small readiness check. The token must not be pasted into a committed file.
- **Anthropic lines:** store the coach's API key in the local conda environment. `run\_skills.py` reads it when the planner is `anthropic`.

Path B only needs the Python packages (no simulator, no GPU) on your machine; the `robotwin` environment above works for both. Path A additionally needs the RoboTwin assets, CuRobo and a working NVIDIA Vulkan driver because the simulator and renderer run locally.

16.2 Path A: run on your own GPU

A1. RoboTwin and its assets. Run from the repository root. The task only needs the `embodiments` and `objects` bundles (∼4 GB); the 11 GB background textures are only used with random backgrounds, which `demo_clean` turns off.

```
git clone https://github.com/RoboTwin-Platform/RoboTwin.git RoboTwin
git -C RoboTwin checkout ea8b21121ebb3cd201ff5b3fe361944ac94eda3f
git -C RoboTwin apply ../patches/robotwin-renderer.patch
cd RoboTwin/assets
HF_HUB_DISABLE_XET=1 python -c "from huggingface_hub import snapshot_download as get; \
   get('TianxingChen/RoboTwin2.0', repo_type='dataset', local_dir='.', \
   allow_patterns=['embodiments.zip', 'objects.zip'])"
unzip -q embodiments.zip && rm embodiments.zip
unzip -q objects.zip && rm objects.zip && rm -rf __MACOSX
cd ..  && python scripts/update_embodiment_config_path.py && cd ..
```

A2. CuRobo (the motion planner). It compiles GPU code, so the compiler settings are stored in the conda environment once. CUDA 12.1's `nvcc` refuses gcc newer than 12; set `TORCH_CUDA_ARCH_LIST` to your GPU (8.6 for an RTX 2050/30-series).

```
git clone --branch v0.7.8 https://github.com/NVlabs/curobo.git RoboTwin/envs/curobo
git -C RoboTwin/envs/curobo checkout d64c4b005459db10c5dd867d8b30a87d5bda9bdb
conda env config vars set CUDA_HOME="$CONDA_PREFIX" CC=/usr/bin/gcc-12 \
   CXX=/usr/bin/g++-12 CUDAHOSTCXX=/usr/bin/g++-12 \
   NVCC_PREPEND_FLAGS="-ccbin /usr/bin/g++-12" TORCH_CUDA_ARCH_LIST=8.6
conda deactivate && conda activate robotwin
python -m pip install -e RoboTwin/envs/curobo --no-build-isolation   # 5-15 minutes
python -m pip install warp-lang==1.12.0
python -c "import curobo; print('curobo ok')"
```

A3. Vulkan: use only the NVIDIA driver. Laptops with a second (Intel) GPU can freeze the renderer otherwise.

```
conda env config vars set VK_DRIVER_FILES=/usr/share/vulkan/icd.d/nvidia_icd.json \
   VK_ICD_FILENAMES=/usr/share/vulkan/icd.d/nvidia_icd.json
conda deactivate && conda activate robotwin
vulkaninfo --summary | grep deviceName    # should list only your NVIDIA GPU
```

A4. Climb the ladder. Tests, then the smoke run, then a full episode with a time limit. `timeout -s INT` sends the same signal as Ctrl+C, so the run still writes `summary.json` and its video.

```
python -m unittest discover -s tests -q
python run_skills.py --config configs/skill_runtime.yaml \
   --development-oracle --output data/runs/smoke_01
timeout -s INT 75m python run_skills.py \
```

```
  --config configs/skill_runtime_opus_full_task_rgbd_table_lift.yaml \
  --planner anthropic \
  --perception-factory harness.perception.modal_sam3:create_detector \
  --output data/runs/full_task_01
```

16.3 Path B: run on Modal

Like I'm five

Instead of using your own computer's graphics card, you **rent** a stronger one for as long as the episode lasts. Modal starts a fresh, clean computer (a *container*) for every episode, runs exactly the same `run_skills.py` there, keeps the results in a cloud folder (a *Volume*), and copies them back to your `data/runs/` folder at the end.

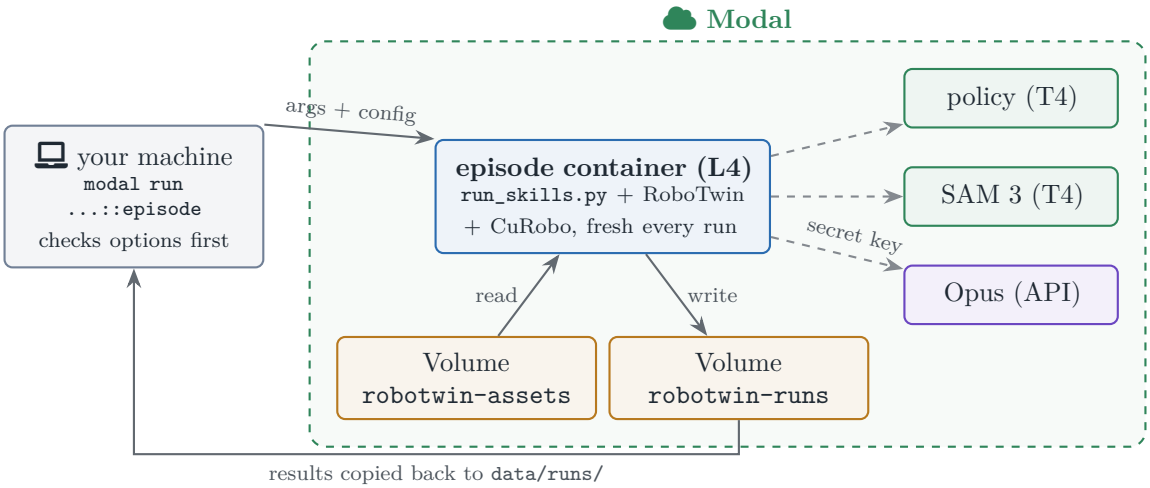


Figure 26: An episode on Modal (</> `modal_apps/simulator.py`). Policy and SAM 3 calls stay inside Modal.

B1. One-time setup. The first command builds the cloud image (RoboTwin, CuRobo for T4/A10G/L4/A100, the same patches as path A; about 20–40 minutes once, then cached) and copies the assets into a Volume. The spike is optional: it measures a GPU and saves a camera picture to `data/modal_spike/`.

```
# needed by every run of simulator.py, even the spike
modal secret create anthropic ANTHROPIC_API_KEY=sk-ant-...
modal run modal_apps/simulator.py::download_assets         # CPU only, once
modal run modal_apps/simulator.py::spike                   # optional:  test the L4
SIM_GPU=T4 modal run modal_apps/simulator.py::spike       # optional:  test a T4
```

B2. Run episodes. Use the normal `run_skills.py` options after `::episode`. Two extra options belong to the launcher: `--time-limit-minutes` (default 90, at most 210) and `--skip-observations` (leave the per-step pictures on the Volume).

```
# rung 3:  cheap smoke run (no Opus calls)
modal run modal_apps/simulator.py::episode \
   --config configs/skill_runtime.yaml --development-oracle \
   --output data/runs/modal_smoke_01 --time-limit-minutes 15

# rung 4:  the full task
modal run modal_apps/simulator.py::episode \
   --config configs/skill_runtime_opus_full_task_rgbd_table_lift.yaml \
   --planner anthropic \
```

```
  --perception-factory harness.perception.modal_sam3:create_detector \
  --output data/runs/modal_full_01 --time-limit-minutes 75
```

B3. Housekeeping.

```
modal volume ls robotwin-runs                              # list remote runs
modal volume get robotwin-runs <run_id> data/runs/     # download a run again
modal volume rm -r robotwin-runs <run_id>             # delete an old run
```

Careful

- **Keep the terminal open**: stopping `modal run` stops the episode. For a run that survives a closed laptop use `modal run --detach ...`, then fetch the results with `modal volume get`.
- **One Modal episode at a time** for now: the policy worker keeps one shared observation history, so parallel episodes would mix their memories.
- Frozen evaluation (`--evaluation-manifest`) is refused on Modal; it stays local for now.

16.4 Running cheaply

Key idea

What is local and what is remote? Your terminal first parses and validates the command. Then `modal\_apps/simulator.py` packages the request and starts a fresh L4 container. That container owns RoboTwin, SAPIEN, CuRobo and the episode loop. It calls the separate T4 policy and SAM 3 helpers, while the Anthropic request goes to the API. At the end, the container copies the run artifacts back to your local `data/runs/` directory. The remote container is temporary; the output directory is the durable record.

Trick	Why it saves
Stay low on the ladder	tests and replays are free; most mistakes show up there
Scripted planner + `--development-oracle`	no Opus calls at all
`--time-limit-minutes` (Modal) or `timeout -s INT` (local)	a stuck run cannot burn hours
`SIM_GPU=T4`	cheaper per hour than the L4; pictures ~3× slower, a full episode only ~1 minute longer
`--skip-observations`	smaller downloads; pictures stay on the Volume
A copied config with smaller `max_turns` / `anthropic.max_calls`	caps Opus turns
Delete old runs from `robotwin-runs`	Volumes keep (and bill for) what you store

Rough cost of one full Modal episode (~30 minutes, mostly waiting for Opus): about $0.30 on a T4 or $0.40 on an L4 of simulator GPU time, plus Opus tokens and policy time. Prices are approximate; check Modal's pricing page and dashboard.

16.5 Measured speed

Same seed, same steps, measured with the spike (`data/modal_spike/`):

	laptop RTX 2050	T4	A10G	**L4**
three RGB-D pictures	800 ms	344 ms	153 ms	**110 ms**
CuRobo plan	322 ms	88 ms	73 ms	86 ms
physics step	0.09 ms	0.01 ms	0.01 ms	0.01 ms
reset (once, cold)	27 s	49 s	33 s	66 s

All three cloud GPUs draw the same picture as the laptop (mean difference 0.03 out of 255), so the policy sees the same thing wherever the episode runs. L4 is the default; T4 is the budget choice.

16.6 When something goes wrong

You see...	Do this
`Permission denied` under `/run/media/...` while downloading	`HF_HOME` points at an unmounted drive. Set it to `\$HOME/.cache/huggingface` or mount the drive.
`Not enough free disk space`	Download only `embodiments` and `objects` (A1); `conda clean -a` frees several GB.
`unsupported GNU version! gcc versions later than 12`	Set the compiler variables in A2 and reactivate the environment.
CuRobo ended up in `RoboTwin/RoboTwin/envs/`	You cloned from inside `RoboTwin/`; run the clone from the repository root.
Episode freezes, GPU at 0 %, log files stop changing	Renderer hang: apply A3, stop with Ctrl+C, rerun with a time limit.
`missing pytorch3d` at start-up	Harmless message from RoboTwin.
Smoke run ends with `step_budget`	Expected (Section 15).
SAM 3: `GatedRepoError ... not in the authorized list`	Your Hugging Face account is not approved yet, or the token is from another account; replace the secret with `--force`.
Modal `ImageBuildError ... already exists; choose a new --output`	Run `modal image logs <id>` with the *newest* id from the error box. Every run needs a new output folder.

16.7 What you get after a run

```
data/runs/full_task_01/
├─ manifest.json          settings, seed, prompt hash
├─ assessments.jsonl      every planner choice and turn result
├─ skill_events.jsonl     every action, tick and check
├─ simulator_timing.jsonl how long RoboTwin took
├─ oracle.jsonl           private evidence (oracle mode only)
├─ observations/          saved pictures + depth (.npz)
├─ episode.mp4            video when record_video: true
├─ summary.json           outcome, counts, timings
└─ candidate_catalog.json memory ideas to review (never auto-used)
```

Figure 27: Run artifacts. Exit code 0 means a success outcome such as `acquisition_confirmed` or `task_success`.

16.8 Other ways to run

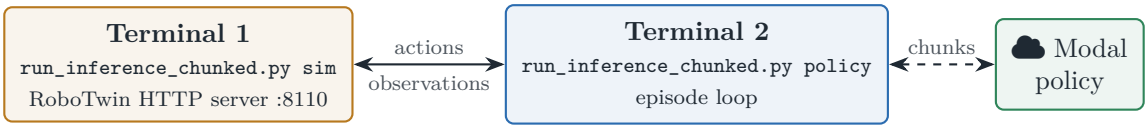


Figure 28: The baseline: the frozen policy alone, no coach. Used to compare against.

- **One episode on Modal** (`modal run modal_apps/simulator.py::episode`) takes the same options as `run_skills.py`; see Section 16.3.
- **Exploration** (`run_exploration.py`) runs many episodes, each in a new process, and collects memory *candidates*. It currently requires `--development-oracle` and runs locally.
- **Evaluation** (`run_evaluation.py freeze/run`) locks seeds, code hashes and memory, so results cannot be accidentally tuned on the test seeds.
- **Offline tools** live in `development/`, for example `python -m development.replays.perception --help`. Most need no simulator.

17 Where things stand today

Part	State	Notes
Baseline policy runner	✔ works	independent of the harness
Skill executor + six skills	✔ works	bounded, tested on CPU fakes
Scripted acquisition / handoff	✔ works	development-oracle evidence
Opus planner, full task	? exploring	phases succeed sometimes; not yet reliable hanging
Public RGB-D perception	? exploring	SAM 3 + Opus grounding; gaps return `null`
Evaluation with perception	✘ not yet	needs a frozen detector/prompt contract
Simulator on Modal	✔ works	T4, A10G and L4 render the same pictures as the laptop
Single episodes on Modal	? new	`::episode` is built and tested offline; first live runs pending
Parallel runs, campaigns on Modal	✘ not yet	needs one policy session per episode
Renderer freeze on two-GPU laptops	? mitigated	pin Vulkan to NVIDIA (Section 16.6)
Unit tests	✔ 435 pass	`python -m unittest discover -s tests`

Careful

- Success of one phase is **not** proof of reliable full-task hanging.
- RoboTwin's native success check is not the same as a verified handle-on-peg hang.
- Configs name the model `claude-opus-5`; check it is the model you intend to use.
- Checkpoints and provenance files recorded before the October 2026 restructure no longer match the current file layout; record new ones.

18 Testing and contributing

- **All tests run on CPU** with fake simulators, fake policies and fake Opus clients. No GPU or API key is needed.
- **Change behaviour, change a test.** New behaviour needs a test that shows it.
- **One subsystem per pull request**, with the contract change and a CPU or replay check.
- **Prompts are code.** Version a prompt change together with the parser/schema it serves.
- **Clean-code defaults** for agents and people are in `.agents/skills/clean-code/SKILL.md`.

A Frequently asked questions

Why not let Opus control the joints directly?
Language models are not precise with numbers and cannot react within milliseconds. Opus judges the situation; deterministic code and the trained policy do the motion.

Why does an unknown answer stop the robot from opening its hand?
Opening a hand that holds an unsupported mug drops it. Without proof, the safe choice is not to open.

What is the "development oracle" for?
Debugging. It uses simulator-only knowledge so we can test skills before perception is good enough. It is never shown to Opus and is not allowed in evaluation with perception.

Why is the policy called in short chunks?
So the coach can look again often. Long chunks are cheaper but react slower.

Where do I start reading code?
`run_skills.py`, then `harness/runtime/assembly.py`, then `harness/runtime/orchestrator.py`.

Why run episodes on Modal instead of my laptop?
A rented L4 draws the camera pictures about seven times faster than a 4 GB laptop GPU, has no memory squeeze, and avoids two-GPU laptop driver trouble. It costs GPU time, so use the ladder.

Do I need RoboTwin on my machine to use Modal?
No. The simulator lives in the cloud image. Your machine only needs the Python packages to check the options and to talk to Modal.

Does running on Modal change the results?
The pictures match the laptop's almost exactly (0.03 out of 255 on average). Still treat local and Modal runs as separate conditions when you compare numbers carefully.

Why only one Modal episode at a time?
The deployed policy remembers recent pictures for the episode it is serving. Two episodes at once would share that memory. Giving each episode its own policy session is the next step.